1) What is a design pattern?

A **design pattern** is a reusable solution to a commonly occurring design problem in software. Patterns are **not** code you copy/paste — they are templates and ideas you adapt. They help make systems easier to understand, maintain, and extend.

---

2) Pattern categories

- **Creational** — control object creation (e.g., Singleton, Factory, Builder, Prototype).
- **Structural** — compose classes/objects into larger structures (e.g., Adapter, Decorator, Facade, Proxy, Composite).
- **Behavioral** — object interaction and responsibilities (e.g., Strategy, Observer, Command, Chain of Responsibility, State, Template Method).

---

3) Python idioms vs classical OOP patterns

Python's dynamic features often let you implement patterns more simply:

- Use **functions / callables** instead of full strategy classes.
- Use **decorators** (language feature) for the Decorator pattern.
- Use **modules** (singletons by nature) as a simple Singleton.
- Use **metaclasses** when you really need a class-level control of instantiation.

Patterns still help with architecture, clarity, and communication. Think of them as vocabulary.

---

4) Short, practical examples

Singleton — single instance

Simple Python approach using a module (recommended) or a class with a metaclass.

Module approach: create config.py and import it — only one module instance exists.

Class approach (metaclass):

```
class SingletonMeta(type):
    _instances = {}
```

```
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Logger(metaclass=SingletonMeta):
    def __init__(self):
        self.records = []

logger1 = Logger()
logger2 = Logger()
assert logger1 is logger2
```

When to use: shared resources (but prefer dependency injection or module-level singletons).

Factory Method / Simple Factory

Encapsulate object creation so calling code doesn't depend on concrete classes.

```
class Dog:
    def speak(self): return "woof"

class Cat:
    def speak(self): return "meow"

def animal_factory(kind: str):
    if kind == "dog": return Dog()
    if kind == "cat": return Cat()
    raise ValueError(kind)
```

```
pet = animal_factory("dog")
print(pet.speak())  # "woof"
```

When to use: hide concrete classes from client, choose subclass at runtime.

Abstract Factory

Return families of related objects.

```
class WinButton: pass
class MacButton: pass

class GUIFactory:
    def create_button(self): raise NotImplementedError

class WinFactory(GUIFactory):
    def create_button(self): return WinButton()

class MacFactory(GUIFactory):
    def create_button(self): return MacButton()

def get_factory(os_name):
    return WinFactory() if os_name == "win" else MacFactory()
```

Builder

Construct complex objects step-by-step — useful for building configurations.

```
class Pizza:
    def __init__(self): self.toppings = []
```

```
    def __repr__(self): return f"Pizza({self.toppings})"

class PizzaBuilder:
    def __init__(self): self.pizza = Pizza()
    def add_cheese(self): self.pizza.toppings.append("cheese"); return self
    def add_pepperoni(self): self.pizza.toppings.append("pepperoni"); return self
    def build(self): return self.pizza

pizza = PizzaBuilder().add_cheese().add_pepperoni().build()
```

Adapter

Make incompatible interfaces work together.

```
class OldPrinter:
    def print_old(self, text): print("Old:", text)

class PrinterAdapter:
    def __init__(self, old_printer): self.old = old_printer
    def print(self, text): self.old.print_old(text)

old = OldPrinter()
adapter = PrinterAdapter(old)
adapter.print("hello")  # works with new interface
```

Decorator (structural) — using Python decorators

Wrap behavior without changing the object.

Function decorator example:

```
def timed(fn):
    import time
    def wrapper(*args, **kwargs):
        start = time.time()
        result = fn(*args, **kwargs)
        print(f"{fn.__name__} took {time.time()-start:.6f}s")
        return result
    return wrapper

@timed
def compute(n):
    s = 0
    for i in range(n): s += i
    return s
Object decorator (wrap instance):
class Component:
    def operation(self): return "component"

class LoggingDecorator:
    def __init__(self, comp): self.comp = comp
    def operation(self):
        print("before")
        result = self.comp.operation()
        print("after")
        return result
```

Proxy

Control access / lazy loading.

```
class Image:
    def __init__(self, path): self.path = path
    def display(self): print(f"Displaying {self.path}")


class LazyImageProxy:
    def __init__(self, path):
        self.path = path
        self._real = None
    def display(self):
        if self._real is None:
            self._real = Image(self.path)   # create only when needed
        self._real.display()
```

Strategy

Encapsulate algorithms and swap them at runtime. In Python, functions are great strategies.

```
def sort_asc(data): return sorted(data)
def sort_desc(data): return sorted(data, reverse=True)


class SortContext:
    def __init__(self, strategy):
        self.strategy = strategy
    def sort(self, data): return self.strategy(data)


ctx = SortContext(sort_desc)
```

```
print(ctx.sort([3,1,2]))  # [3,2,1]
```

Observer (Publish/Subscribe)

Objects subscribe to events; publisher notifies them.

```
class Subject:
    def __init__(self): self._observers = []
    def register(self, obs): self._observers.append(obs)
    def notify(self, msg):
        for o in self._observers: o.update(msg)

class Observer:
    def update(self, msg): print("got", msg)

s = Subject()
o = Observer()
s.register(o)
s.notify("hello")
```

In real apps prefer asyncio events, signal-like libraries, or pub/sub systems for scale.

Command

Encapsulate a request as an object (useful for undo/redo, queues).

```
class Command:
    def execute(self): raise NotImplementedError

class AddCommand(Command):
    def __init__(self, receiver, value): self.receiver = receiver; self.value = value
```

```
    def execute(self): self.receiver.add(self.value)


class Receiver:
    def __init__(self): self.data = []
    def add(self, v): self.data.append(v)


r = Receiver()
cmd = AddCommand(r, 5)
cmd.execute()
```

---

5) When to use (practical guidance)

- Use patterns to **communicate design** to others (naming matters).
- Prefer **simple code** first. Only apply a pattern when it solves a real problem (YAGNI).
- Favor Pythonic idioms: functions, modules, context managers, generators — they often replace heavyweight patterns.
- Use patterns for: extensibility, testability, decoupling, swapping implementations, and simplifying complex creation logic.

---

6) Common anti-patterns and pitfalls

- **Overuse / premature abstraction** — don't create layers you don't need.
- **Singleton abuse** — global state hurts testability. Prefer dependency injection.
- **Rigid inheritance trees** — prefer composition over inheritance.
- **Complex patterns for small projects** — keep it simple.

---

7) Testing & patterns

- Patterns improve testability if they **decouple** code (e.g., Strategy, Factory).
- Use mocks/stubs for collaborators; design for injection of dependencies.

- Keep side effects isolated to make unit tests deterministic.

---

8) Quick cheatsheet — when to pick a few common patterns

- Need controlled single instance? → Module or Singleton.
- Need different algorithms at runtime? → Strategy.
- Need to observe state changes? → Observer (or events).
- Need to add behavior to objects without change? → Decorator.
- Need to hide complex subsystem? → Facade.
- Need to create objects without exposing class names? → Factory / Abstract Factory.
- Need to queue & execute commands? → Command.
